

FORGE — FDE Academy · D4 · 26-Jun-2026



Python for AI Scaffolding

Type-safe contracts · resilient clients · function-calling · FastAPI

The scenario (today's spine)

- It works in **staging**. In **production** the same model returns the same meaning in a different JSON shape — and your code crashes in front of the client.
 - staging: `{"text": ..., "finish_reason": ...}`
 - prod: `{"message": {"content": ...}, "stop_reason": ...}`
- The model didn't break — your **contract assumption** broke.

What you'll be able to do

- Write async, typed, packaged Python for AI calls — not notebook scripts
- Use strict **Pydantic v2** models as the LLM↔app contract
- Build typed clients with **retries + backoff**
- Validate **function-calling** args and return typed errors
- Expose it as a **FastAPI** service with an OpenAPI `/docs`
- Keep the provider swappable (**ports & adapters**) and pin it with **contract tests**

Mental model: the model is a flaky vendor

- Treat every model call like a network call to an unreliable third party:
 - **Type it** — strict Pydantic at the boundary
 - **Validate it** — reject garbage at the door, with the field name
 - **Retry it** — transient failures (429/5xx) with backoff
 - **Time it out** — never hang the event loop
- Everything today is one of those four verbs.

Python for AI — async + packaging

- LLM calls are **I/O-bound**: one slow call must not block the event loop
- `async` lets calls overlap → one FDE serves concurrent users on a demo
- `uv` / venv = reproducible installs; every lab repo resolves identically on every VM

```
async def complete(req):  
    raw = await provider.complete(req)  
    return CompletionResponse.model_validate(raw)
```

Pydantic v2 = contract, not boilerplate · TODO 1

- ``strict=True`` rejects ``prompt=123`` instead of silently casting to `"123"`
- ``@model_validator(mode="before")`` runs **before** fields — absorb drift here
- Map **both** shapes to one clean `{text, model, finish_reason}``
- Delete the validator live → `ValidationError` at the boundary, not a silent `KeyError`

```
@model_validator(mode="before")
def normalize(cls, d):
    if "message" in d:
        return {"text": d["message"]["content"],
                "finish_reason": d.get("stop_reason")}
    return d
```

Typed clients + retries · TODO 2

- Wrap the provider in a typed client; validate inside the loop (the boundary)
- **TransientError = retry · PermanentError = give up**
- Exponential backoff `base * 2^attempt` (add jitter in prod)
- Provider SDKs already retry 429/5xx — wrap that, don't reinvent it

```
for attempt in range(max_retries + 1):  
    try:  
        return CompletionResponse.model_validate(  
            await provider.complete(req))  
    except TransientError:  
        await asyncio.sleep(base * 2**attempt)
```

Ports & adapters = provider freedom

- "Procurement **will** change your model vendor mid-engagement — not if, when."
- Azure OpenAI is the deploy target (migrated 06-Jul)
- The adapter you write today is exactly what gets re-pointed then
- **No SDK import belongs in business logic** — make the swap one line

Function calling = contract negotiation · TODO 3

- The model hands you a JSON **string** of args — ``json.loads`` first, then validate
- It sometimes breaks the contract: bad JSON, wrong types, extra fields
- Return a **typed error** (``ToolCallResult(ok=False, ...)``), never a stack trace

```
try:
    args = tool_model.model_validate(json.loads(raw["arguments"]))
except (JSONDecodeError, ValidationError) as e:
    return ToolCallResult(ok=False, error=TypedError(
        code="bad_tool_args", message=str(e), retryable=False))
```

FastAPI + /docs · the 422 is a feature

- `response\_model` advertises the typed schema and validates on the way out
- Submit an integer `prompt` to `/docs` → **422** with the exact field error
- Show the 422 as strict mode **working**, not a bug

```
@app.post("/complete", response_model=CompletionResponse)
async def complete(req: CompletionRequest):
    return await client.complete(req)
```

Contract tests = the handoff artefact

- A green contract test pins this boundary for every later session
- 29-Jun (DDD) and 02-Jul (integration) **trust** this test is green
- It fails loudly on the prod-shaped JSON, then passes once normalized

Your labs — 3 TODOs in order

- Open ``trainee-kit/lab/starter``, run ``./setup.sh``, see red — that's the spec
 - 1) ``app/models.py`` — normalize drift
 - 2) ``app/client.py`` — retry/backoff + validate
 - 3) ``app/function_calling.py`` — parse + typed errors
- "Read the failing test name — it tells you exactly what to build."
- Green = ``20 passed, 1 skipped`` → start the API → break it on ``/docs``



Build the scaffold every later session stands on.

pytest green · contract tested · provider swappable · committed for 29-Jun